

Software Testing Lab —

Detecting and Correcting Software Faults

MODULE	Software Testing and Quality Assurance
LAB TOPIC	Identification, Analysis, and Correction of Software Faults
LANGUAGE	Java

PREPARED BY

Mohamed Yacine Saadia

PROMOTION Master 2 Software Engineering

1. Introduction

Software development is not limited to writing code that compiles and executes. A software application must also satisfy its requirements, produce correct results, protect user data, respond efficiently, and provide a good user experience.

However, developers can introduce faults into their code. These faults may cause incorrect results, unexpected behavior, security vulnerabilities, performance problems, or even application crashes.

The objective of this lab is to understand how software faults affect software quality and how we can identify, analyze, and correct them.

1.1. Errors, faults, bugs, and failures

These terms are related but do not have exactly the same meaning.

Error: A human mistake made during software development, such as misunderstanding a requirement or writing an incorrect condition.

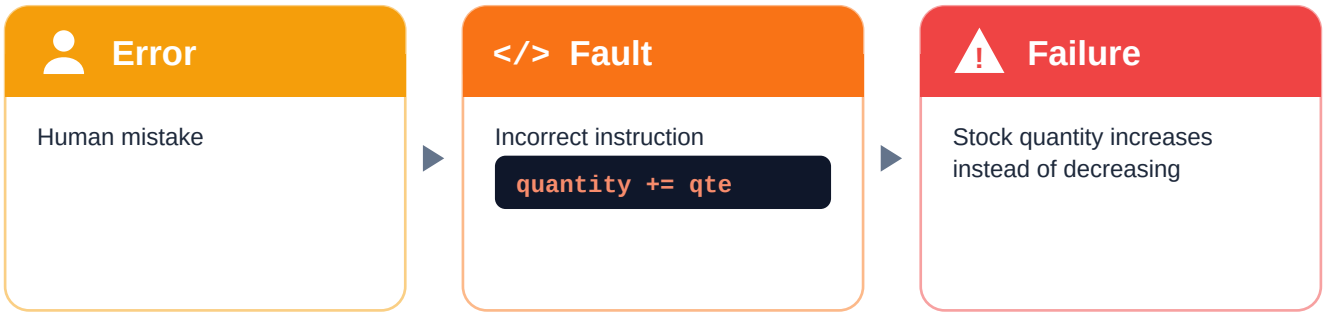
Fault (defect): An incorrect instruction, condition, or design decision introduced into the software because of an error.

Bug: An informal term commonly used to describe a software defect.

Failure: An observable event in which the software does not behave as required during execution.

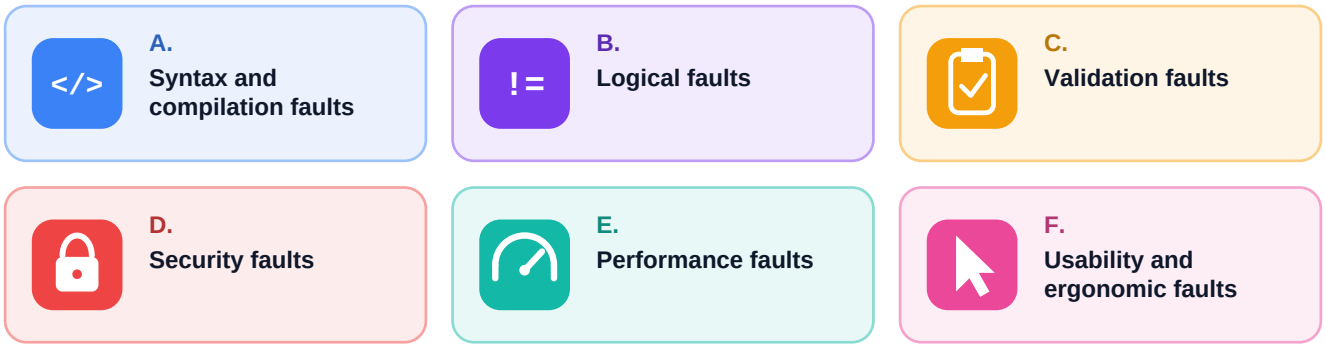
For example, a developer writes `quantity += qte` instead of `quantity -= qte` in a product purchasing method.

The developer has made an error, the incorrect instruction is a fault, and the stock quantity increases instead of decreasing when the program executes. This incorrect behavior is a failure.



A fault does not necessarily produce a failure every time the program executes. The conditions required to activate it may not always occur.

1.2. Types of software faults



A. Syntax and compilation faults

These faults prevent the program from compiling or running correctly.

Example:

```
public private double solde;
```

The declaration is invalid because two access modifiers are used together.

B. Logical faults

The program executes, but its behavior does not satisfy the requirements.

Example:

```
public void acheter(int qte) {
    quantity += qte;
}
```

If purchasing a product should reduce its stock, the operation is incorrect.

C. Validation faults

The program accepts invalid input or fails to check business rules.

Example:

```
public void deposer(double montant) {
    solde += montant;
}
```

A negative deposit can decrease the account balance.

D. Security faults

These faults may expose confidential information or allow unauthorized operations.

Example:

```
public double solde;
```

Making a sensitive field public allows external code to modify it directly, potentially bypassing the class's business rules.

E. Performance faults

These faults cause the application to consume excessive time, memory, or processing resources.

Example: repeatedly querying a database inside a loop when one query could retrieve all the required information.

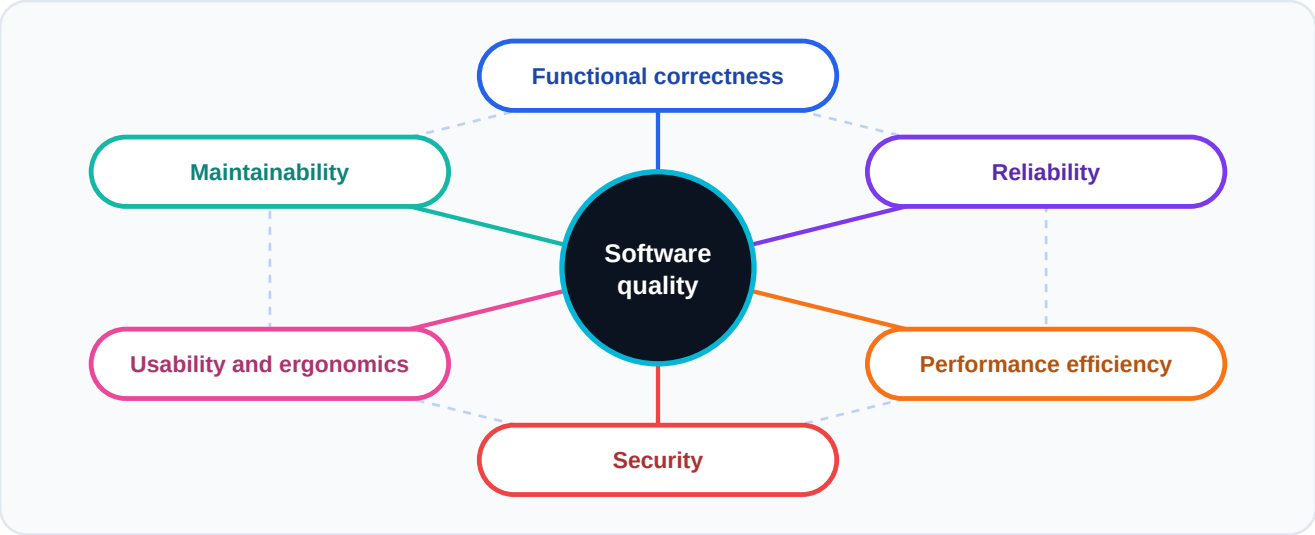
F. Usability and ergonomic faults

These faults make the software difficult, confusing, or uncomfortable to use.

Examples include unclear error messages, confusing navigation, unreadable text, and a confirmation button placed where users are likely to click the wrong action.

1.3. How faults affect software quality

Software quality can be evaluated through several characteristics and measurable indicators. The following examples illustrate how faults can affect them.



Quality characteristic	Example metric	Example of a fault's impact
Functional correctness	Percentage of tests passed; number of incorrect results	A purchase increases stock instead of decreasing it.
Reliability	Failure rate; mean time between failures (MTBF)	A fault causes the application to crash during certain operations.
Performance efficiency	Response time; memory usage; CPU usage	An inefficient algorithm makes a page load too slowly.
Security	Number and severity of vulnerabilities; unauthorized access attempts	A user can modify an account balance without authorization.
Usability and ergonomics	Task completion rate; task completion time; user error rate	Users repeatedly select the wrong button because of confusing interface design.
Maintainability	Time required to modify code; code complexity; duplicated code	Poorly structured code makes corrections difficult and risky.

These metrics help us evaluate different aspects of quality. A program can be fast but insecure, easy to use but functionally incorrect, or correct in normal situations but unreliable in exceptional cases.

> ⚠ **Important:** Passing compilation does not prove that a program is correct. Likewise, passing a few tests does not guarantee that every fault has been detected.

2. Lab Objectives

At the end of this lab, students should be able to:

- ✓ Distinguish compilation errors from logical faults.
- ✓ Identify faults through code inspection and program execution.
- ✓ Compare actual behavior with expected behavior.
- ✓ Recognize invalid inputs and boundary-condition problems.
- ✓ Explain how faults affect software quality.
- ✓ Correct faulty code without introducing new defects.
- ✓ Propose unit tests to verify the corrected behavior.

3. General Instructions

For each exercise:

1. Read the code carefully before executing it.
2. Identify any compilation errors.
3. If the program compiles, execute it and observe its behavior.
4. Compare the actual results with the expected business rules.
5. Identify and classify the faults.
6. Correct the faults and justify your modifications.
7. Propose test cases to verify the corrections.

> ⛔ Do not modify the code immediately. First, try to understand what the program is supposed to do and determine why its current behavior is incorrect.

Exercise 1 — Bank Account Management

Context: A bank is developing a simple application to manage customer accounts. Each account has a balance and supports deposits and withdrawals.

The application must preserve the integrity of the account balance and reject invalid operations.

Part A — Analyze the faulty code

The following code contains faults. Identify them before attempting to correct the program.

File: `Compte.java`

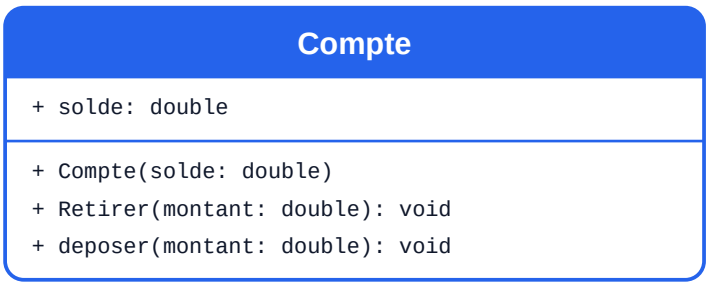
```java
public class Compte {
    public double solde;
    public Compte(double solde){
        this.solde=solde;
    }
    public void Retirer(double montant) {
        if (solde - montant > 0) {
            solde -= montant;
        }
        System.out.println(solde);
    }

    public void deposer(double montant) {
        solde += montant;
        System.out.println(solde);
    }
}

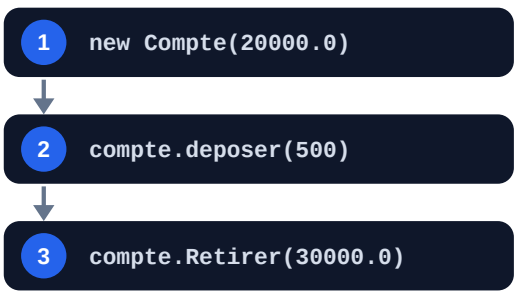
//Test Case 1//
void main() {
    Compte compte=new Compte(20000.0);
    System.out.println("the initial sold is "+ compte.solde);
    compte.deposer(500);
    compte.Retirer(30000.0);


}
```

CLASS DIAGRAM



EXECUTION SCENARIO



the actual correction table

Public solde field: anyone can write c.solde = -1000	`c.solde = -1000`	Impossible: the field is private, access goes through the methods
> instead of >=: the account cannot be emptied	`balance 100, retirer(100)`	Balance 0
Silent failure: a refused withdrawal produces no error, the caller believes the money was withdrawn	`balance 50, retirer(80)`	Exception to be thrown
deposer accepts a negative, zero or NaN amount: the positive-balance rule is bypassed	`deposer(-500)`	Rejected
double for money: rounding errors	`10 deposits of 0.1 give 0.9999999999999999`	BigDecimal (or integer cents)
No validation in retirer: a negative amount increases the balance	`retirer(-100)`	Rejected

Exercise 2 — Product and Stock Management

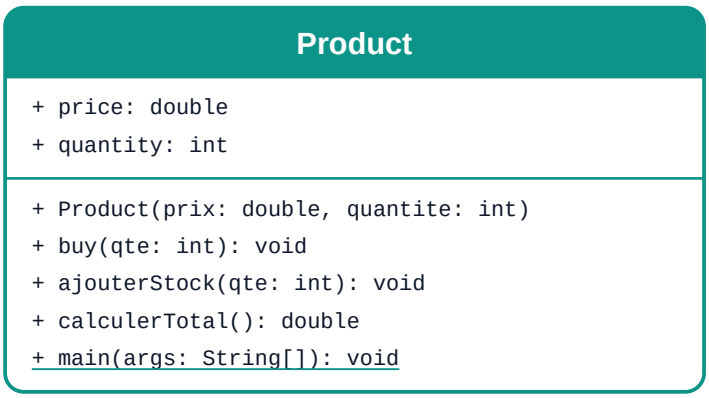
Context: A company is developing an inventory management application. Each product has a price and a quantity in stock.

The application must allow users to purchase products, add new stock, and calculate the total value of the current stock.

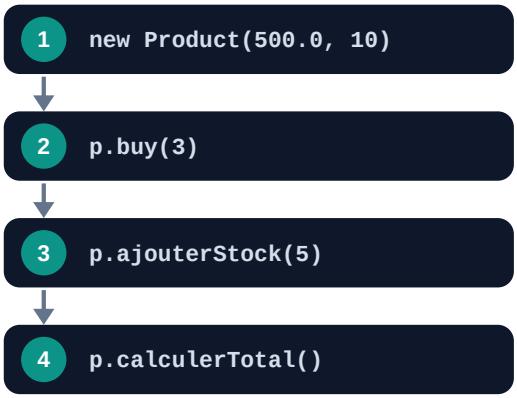
Part A — Analyze the faulty code

The following Java code contains several faults related to initialization and stock management.

CLASS DIAGRAM



EXECUTION SCENARIO



File: `Product.java`

```java
package org.example;

public class Product {
    public double price;
    public int quantity;

    public Product(double prix, int quantite) {
        this.price = price;
        this.quantity = quantity;
    }

    public void buy(int qte) {
        if (quantity >= qte) {
            quantity += qte;
        }

        System.out.println(
            "Quantite restante: " + quantity
        );
    }

    public void ajouterStock(int qte) {
        quantity -= qte;

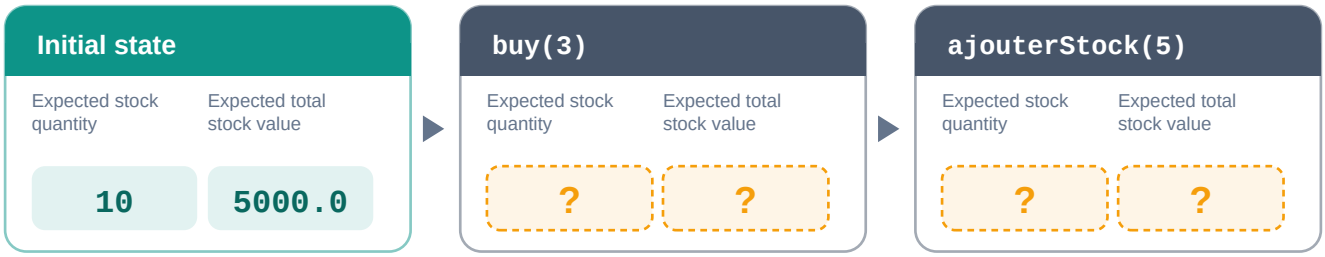
        System.out.println(
```

```
26                "Nouveau stock: " + quantity
27            );
28        }
29
30        public double calculerTotal() {
31            return price * quantity;
32        }
33
34        public static void main(String[] args) {
35            Product p = new Product(500.0, 10);
36
37            System.out.println("Prix initial: " + p.price);
38            System.out.println("Stock initial: " + p.quantity);
39
40            p.buy(3);
41            p.ajouterStock(5);
42
43            System.out.println("Total: " + p.calculerTotal());
44        }
45    }
```

Part B — Predict the behavior

Before correcting the code, complete the following table with the behavior of the intended inventory system.

The initial product should have a price of `500.0` and a stock quantity of `10`.



Operation	Expected stock quantity	Expected total stock value
Initial state	**10**	**5000.0**
`buy(3)`	?	?
`ajouterStock(5)`	?	?

Then compare these expected values with the results produced by the faulty program.

Part C — Correction and testing

1. Correct the constructor so that it initializes both attributes using its parameters.
2. Correct `buy(int qte)` so that purchasing products decreases the available stock.
3. Correct `ajouterStock(int qte)` so that replenishing stock increases the available quantity.
4. Add validation for zero and negative quantities.
5. Prevent stock quantities from becoming negative.
6. Consider whether `price` and `quantity` should be public or private.
7. Correct `calculerTotal()` if necessary.
8. Propose at least six test cases, including successful purchases, insufficient stock, stock replenishment, invalid quantities, and total-value calculations.

4. Final Discussion

After completing both exercises, discuss the following topics:

1. The difference between a fault and a failure.
2. The difference between compilation errors and logical faults.
3. The impact of account-management and stock-management faults on users and business operations.
4. The software quality characteristics affected in each exercise.
5. The role of unit testing in detecting faults before delivery.
6. The importance of testing invalid inputs and boundary conditions, rather than only normal scenarios.

5. Expected Deliverables

For each exercise, submit:

- [ ] The list of identified faults and their classifications.
- [ ] The expected behavior and the actual behavior of the original program.
- [ ] The corrected Java source code.
- [ ] The test cases and their expected results.
- [ ] A short explanation of how the identified faults affected software quality.

Conclusion: Software quality depends not only on whether a program compiles, but also on whether it behaves correctly, handles exceptional situations, protects its data, performs efficiently, and remains usable and maintainable. Identifying and correcting faults is an essential part of producing reliable software.